

Working of JS Engine

1. What is JavaScript Engine?

- a. A JavaScript engine is simply a computer program that executes JavaScript code.
- b. It's responsible for translating human-readable JavaScript code into machine-readable instructions that the computer's hardware can execute.
- c. Every browser has its own JS engine, but the most well known one is **Google's v8 Engine**.
- d. This v8 Engine powers Google Chrome and also Node.js (used to build server side applications).

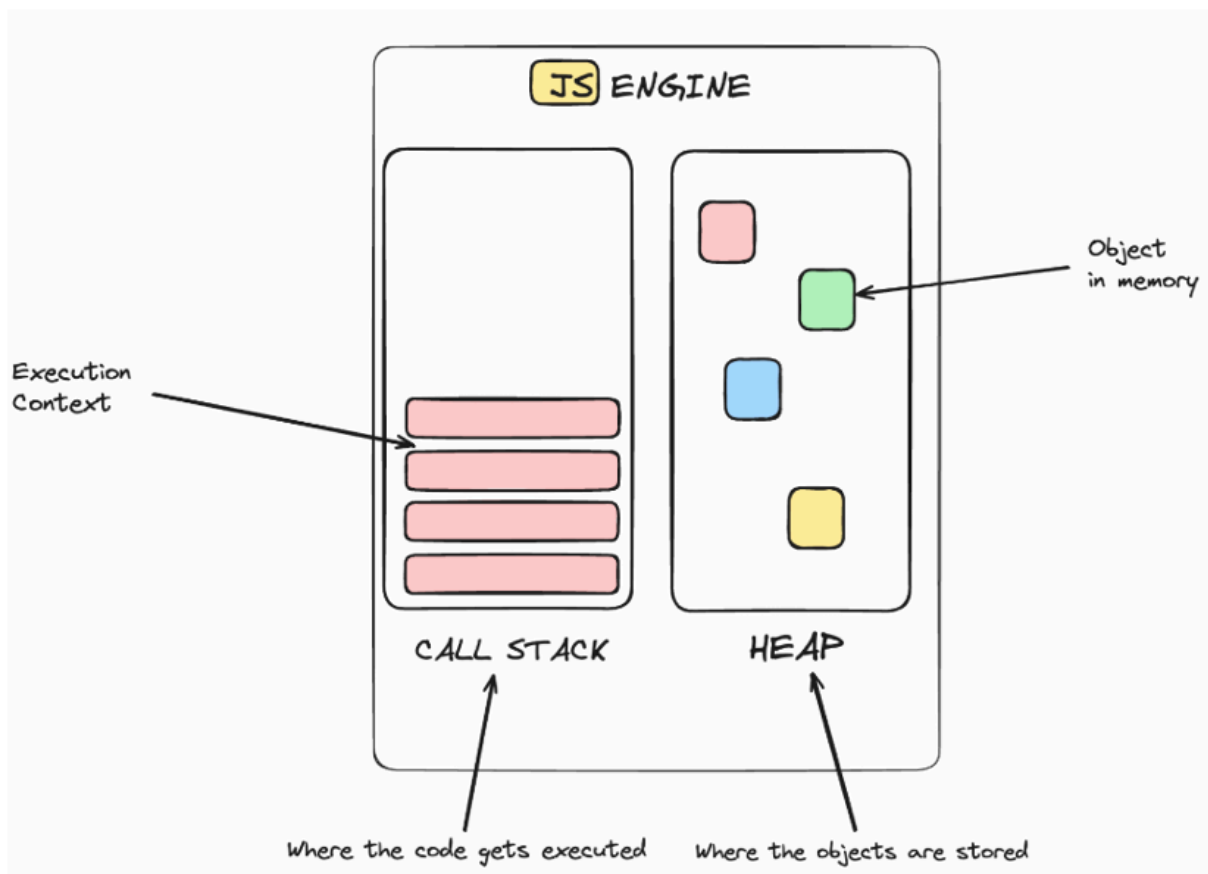
2. List of JavaScript Engines

- a. Here is a list of JavaScript Engines for major Internet browsers:
 - i. **V8** → Google Chrome
 - ii. **SpiderMonkey** → Mozilla Firefox
 - iii. **JavaScriptCore** → Safari
 - iv. **Chakra** → Microsoft Edge
 - v. **JerryScript** → A JavaScript Engine employed for the Internet of Things IoT

3. What are the components of JavaScript Engine?

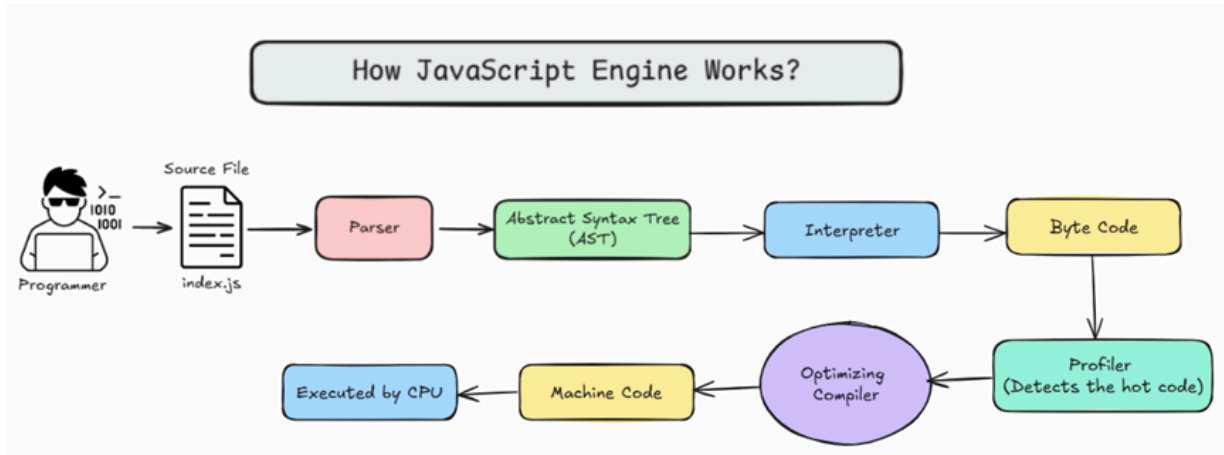
- a. The Engine consists of two main components:
- b. JavaScript Engine consists of a call stack and a heap.

- i. **Memory Heap** → this is where the memory allocation happens. Stores all dynamic data like objects, arrays, and function definitions.
- ii. **Call Stack** → this is where your stack frames are as your code executes. Tracks the execution of functions using a **Last In, First Out (LIFO)** principle. Stores the **execution context** of each function, including local variables and primitive values (like numbers and booleans).



4. How does the JavaScript engine work?

- Following diagram explains how our JS file will be executed in the browser's JS engine:



1. Programmer / Source File ([index.js](#)):

- This is the **starting point** where you, the developer, write your JavaScript code in files like **`index.js`**.
- The JavaScript engine's job is to take this human-readable code and make it understandable to the CPU, which only understands machine code (binary instructions).

2. Parser:

- The parser's role is to read your JS code line-by-line and convert it into a tree structure known as the **AST** (Abstract Syntax Tree).
- It checks for **syntax errors**.
- Example:
`let x=5; //gets broken down into nodes: VariableDeclaration, Identifier, Literal, etc`

3. Abstract Syntax Tree (AST):

- a. AST is a *tree-like structure* that represents the logical structure of your code.
- b. It helps the engine understand the meaning of your program (not just what it says, but what it does).
- c. The AST is passed to the Interpreter next.

4. Interpreter:(Ignition in v8)

- a. The interpreter takes the AST and quickly converts it into bytecode (an intermediate low-level representation).
- b. Bytecode is faster to execute than source code and is portable across different machines.

5. ByteCode:

- a. Bytecode is an intermediate format—more optimized than source code, but not yet machine code.
- b. OR
- c. Bytecode is an intermediate code between source code and the machine code.

6. Profiler:

- a. It detects the hot code.
- b. It identifies "*hot code*"—frequently used functions or loops that are good candidates for optimization.
- c. Example: If a *for loop* is running thousands of times, the profiler will mark it for optimization.

7. Optimizing Compiler: (TurboFan in v8)

- a. This compiler takes the hot bytecode and compiles it into highly optimized machine code.
- b. This stage greatly boosts performance for frequently-run code.

8. Machine Code:

- a. This is the final version of your code, compiled into native instructions that the CPU can directly understand.
- b. Much faster than interpreting bytecode.

9. Executed by CPU:

- a. Finally, the machine code is run directly by the CPU.

10. Summary of the process:

1. JavaScript Code (index.js)
↓
2. Parser → Converts code into AST
↓
3. AST → Interpreter turns into Bytecode
↓
4. Profiler finds hot functions
↓
5. Optimizing Compiler turns bytecode into Machine Code
↓
6. CPU Executes the Machine Code

5. Environment Setup

a. VS Code →

- i. Download the installer for your operating system from the Visual Studio Code website.
- ii. Click on Download Option.
- iii. Install the Software.
- iv. Install Required Extensions like live server etc.

b. Node JS →

- i. Go to the Node.js website.
- ii. Download and install the LTS (Long-Term Support) version.
- iii. Verify the installation by opening a terminal or command prompt and typing: `node -v`
- iv. To run JS file type `node filename.js` in terminal.